Week 9 – Project Work

Event-Driven Orders with Pub/Sub (Pizza Orders)

What I did this week

This week I worked on the main flow of the project. Orders go to a Pub/Sub topic called "orders", then a Cloud Function picks them up and writes them to Firestore. I wanted to make sure that even if the function fails or is slow, the orders don't get lost. So I tested both a normal successful order and a bad one that should fail.

How the flow works (simple version)

Order comes in → published to Pub/Sub topic "orders" → Cloud Function "processOrder" runs → if everything is ok it writes the order to Firestore. If something is wrong it throws an error and Pub/Sub will retry a few times. After that it goes to a dead letter topic.

Test 1 – Successful order

I sent a normal order with orderId, customer name, items and total. The function accepted it and wrote it to Firestore. Here is what the logs looked like (from my local test of the same logic):

```
[Pub/Sub] Published message to topic "orders"
[Cloud Function] Invoked
[Firestore] Wrote document orders/ord-20261010-001
[Cloud Function] Successfully processed order ord-20261010-001
Result: success
```

The order is now in Firestore with status "received". Even if the kitchen system is slow later, this order is already saved and won't be lost.

Test 2 – Failed order (bad data)

Then I tried sending an order without the orderId field on purpose. The function should reject it. This is what happened:

```
[Pub/Sub] Published message to topic "orders"
[Cloud Function] Invoked
[Cloud Function] Invalid payload – missing orderId
Result: Failed – Invalid payload: orderId is required
(In real system this would be retried a few times then go to dead letter topic)
```

This is the expected behavior. Bad messages should not be written to the database. They get retried and eventually moved to the dead letter topic so we can check them later.

Extra check – same order sent twice

I also published the same successful order again to see if it creates a duplicate. The function saw that the document already exists and skipped the write. So no duplicate orders. This is important because Pub/Sub can deliver the same message more than once.

Things I paid attention to

- Used orderId as the document ID so the same order can't be written twice
- Function checks if the document already exists before writing
- Invalid messages are rejected early
- Planned for retries and a dead letter topic
- Service account should only have the minimum permissions needed

---

Note: The logs above are from testing the function logic locally with the same code I plan to deploy. I haven't deployed everything to a live GCP project yet for this weekly sample, but the behavior matches what the real services will do.